

Complete Textbook Notes on DSA algos

Stacks and Queues • Verbatim Conversation Edition

TEXTBOOK EDITION

Professional study and reference edition

Original conversation preserved verbatim in the transcript section.

Prepared as a print-ready DSA study document

Table of Contents

1. Linear Data Structures
2. Stack
3. Stack Terminology
4. Stack Operations
5. Stack Example
6. Stack Overflow
7. Stack Underflow
8. Implementing Stack Using an Array
9. Stack Using a Fixed-Size Array
10. Array Stack Implementation
11. Stack Using Linked List
12. Applications of Stack
13. Stack and Function Calls
14. Recursion and Stack
15. Balanced Parentheses
16. Invalid Parentheses
17. Valid Parentheses Algorithm
18. Monotonic Stack
19. Why Monotonic Stacks?
20. Next Greater Element
21. Queue
22. Real-World Queue
23. Queue Operations
24. Queue Example
25. Queue Overflow
26. Queue Underflow
27. Queue Using Python
28. Why Not Use a Python List for Queue?
29. Circular Queue
30. Circular Queue Formula
31. Queue Using Linked List
32. Applications of Queue
33. BFS and Queue
34. BFS Implementation
35. Deque
36. Stack vs Queue
37. Stack vs Queue Visualization
38. Important DSA Pattern: Stack vs Queue
39. Complexity Cheat Sheet
40. Common Interview Problems — Stack
41. Common Interview Problems — Queue
42. Stack Using Two Queues
43. Queue Using Two Stacks

- 44. Amortized $O(1)$
- 45. Monotonic Queue
- 46. Key Mental Models
- 47. Stack Problem Recognition
- 48. Queue Problem Recognition
- 49. Most Important Things to Memorize
- 50. Final Cheat Sheet

Supplementary textbook material

Chapter summaries, revision notes, glossary, index, and visual learning aids are presented after the verbatim transcript. They are additional material and do not replace or modify the original conversation.

Part I — Verbatim Conversation

The following section contains the conversation exactly as it appears in this chat. Markdown markers, code, examples, formulas, tables, and wording are retained.

USER

Give me the detailed notes on stacks and queues in detail

ASSISTANT

Stacks and Queues — Detailed DSA Notes

Stacks and queues are linear data structures that organize elements according to specific rules for insertion and removal.

They are extremely important for DSA interviews because they appear directly in problems involving parentheses, monotonic stacks, recursion, BFS, scheduling, caching, parsing, and expression evaluation.

1. Linear Data Structures

A linear data structure stores elements in a sequential arrangement.

Examples:

- Array
- Linked List
- Stack
- Queue

The major difference between them is how elements are accessed, inserted, and removed.

Two particularly important restricted linear structures are:

Data Structure	Insertion	Removal	Principle
Stack	One end	Same end	LIFO
Queue	Rear	Front	FIFO

2. Stack

2.1 Definition

A Stack is a linear data structure in which insertion and deletion take place at the same end, called the top.

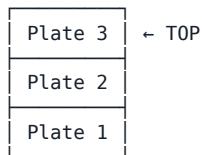
The fundamental principle is:

LIFO — Last In, First Out

The element inserted last is removed first.

Real-world analogy

Think about a stack of plates:



If you want to remove a plate, you remove Plate 3 first.

You cannot directly remove Plate 1 without removing the plates above it.

3. Stack Terminology

A stack primarily has three operations:

1. Push

Adds an element to the top.

Before:

```
TOP
↓
[20]
[10]
```

Push(30)

After:

```
TOP
↓
[30]
[20]
[10]
```

2. Pop

Removes the top element.

Before:

```
TOP
↓
[30]
[20]
[10]
```

Pop()

After:

```
TOP
↓
[20]
[10]
```

30 is removed.

3. Peek / Top

Returns the top element without removing it.

```
TOP
↓
[20]
[10]
```

`peek()` → 20

The stack remains unchanged.

4. Stack Operations

Common stack operations:

Operation	Meaning	Typical Time
<code>push(x)</code>	Insert x	O(1)
<code>pop()</code>	Remove top	O(1)
<code>peek()</code>	View top	O(1)
<code>isEmpty()</code>	Check whether empty	O(1)
<code>size()</code>	Number of elements	O(1)

With a properly implemented stack, all basic operations are generally O(1).

5. Stack Example

Consider:

```
push(10)
push(20)
push(30)
```

Stack becomes:

```
TOP
↓
30
20
10
```

Now:

```
pop()
```

removes 30.

Stack:

```
TOP
↓
20
10
```

Another:

```
pop()
```

removes 20.

Stack:

```
TOP  
↓  
10
```

Therefore:

```
Insertion order: 10 → 20 → 30  
Removal order:  30 → 20 → 10
```

This is LIFO.

6. Stack Overflow

Stack overflow occurs when we attempt to insert an element into a stack that has reached its maximum capacity.

For an array-based stack:

```
Capacity = 3
```

```
[30]  
[20]  
[10]
```

Trying:

```
push(40)
```

causes:

```
STACK OVERFLOW
```

7. Stack Underflow

Stack underflow occurs when we try to remove an element from an empty stack.

```
Stack:
```

```
EMPTY
```

Calling:

```
pop()
```

causes:

```
STACK UNDERFLOW
```

8. Implementing Stack Using an Array

Suppose:

```
stack = []
```

We can use Python's list.

Push

```
stack.append(10)
```

Pop

```
stack.pop()
```

Peek

```
stack[-1]
```

Check empty

```
if not stack:
    print("Stack is empty")
```

Complete example

```
stack = []

stack.append(10)
stack.append(20)
stack.append(30)

print(stack[-1]) # 30

print(stack.pop()) # 30
print(stack.pop()) # 20

print(stack)
```

Output:

```
30
30
20
[10]
```

9. Stack Using a Fixed-Size Array

Conceptually:

```
stack = [_, _, _, _, _]
```

Suppose:

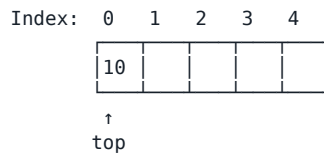
```
capacity = 5
top = -1
```

Initially:

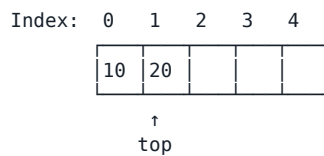
```
Index:  0   1   2   3   4
        [ ][ ][ ][ ][ ]
```

top = -1

After push(10):



After push(20):



10. Array Stack Implementation

```
class Stack:
    def __init__(self, capacity):
        self.capacity = capacity
        self.stack = [None] * capacity
        self.top = -1

    def push(self, value):
        if self.top == self.capacity - 1:
            raise OverflowError("Stack Overflow")

        self.top += 1
        self.stack[self.top] = value

    def pop(self):
        if self.top == -1:
            raise IndexError("Stack Underflow")

        value = self.stack[self.top]
        self.top -= 1
        return value

    def peek(self):
        if self.top == -1:
            raise IndexError("Stack is empty")

        return self.stack[self.top]

    def is_empty(self):
        return self.top == -1
```

Usage:

```
s = Stack(5)

s.push(10)
s.push(20)
s.push(30)

print(s.peek())    # 30
print(s.pop())     # 30
print(s.pop())     # 20
```

11. Stack Using Linked List

A stack can also be implemented using a linked list.

The head can represent the top.

```
TOP
↓
[30] → [20] → [10] → None
```

Push:

Before:

```
TOP
↓
30 → 20 → 10
```

push(40)

After:

```
TOP
↓
40 → 30 → 20 → 10
```

Pop simply removes the head.

This makes push and pop $O(1)$.

12. Applications of Stack

Stacks are used extensively in computer science.

Important applications

1. Function calls
2. Recursion
3. Undo/Redo
4. Browser history
5. Parentheses matching
6. Expression evaluation
7. Infix → Postfix conversion
8. Infix → Prefix conversion
9. DFS
10. Backtracking
11. Monotonic stack problems
12. Syntax parsing
13. Compiler implementation

13. Stack and Function Calls

Consider:

```
def A():
    B()
```

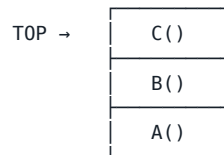
```
def B():
    C()

def C():
    print("Hello")
```

Calling:

```
A()
```

creates a call stack:



C() finishes first.

Then:

```
C() removed
```

Then:

```
B() removed
```

Then:

```
A() removed
```

Again:

LIFO

14. Recursion and Stack

Consider:

```
def factorial(n):
    if n == 1:
        return 1

    return n * factorial(n - 1)
```

Calling:

```
factorial(4)
```

creates:

```
factorial(4)
factorial(3)
factorial(2)
factorial(1)
```

Conceptually:

```
TOP
↓
factorial(1)
factorial(2)
factorial(3)
```

```
factorial(4)
```

Then they return in reverse order.

```
factorial(1)
→ factorial(2)
→ factorial(3)
→ factorial(4)
```

This is why excessive recursion can result in:

Stack Overflow

15. Balanced Parentheses

One of the most important stack applications.

Example:

```
{ [ ( ) ] }
```

We need to determine whether opening and closing brackets match correctly.

Algorithm

Scan left to right.

For every opening bracket:

```
(
[
{
```

push it onto the stack.

For every closing bracket:

```
)
]
}
```

check the top of the stack.

Example:

```
{ [ ( ) ] }
```

Processing:

```
{ → push
[ → push
( → push
) → pop (
] → pop [
} → pop {
```

At the end:

```
Stack = empty
```

Therefore valid.

16. Invalid Parentheses

Consider:

```
{ [ ( ) ] }
```

Processing:

```
{ → push
[ → push
( → push
] → expected [
```

But top is:

```
(
```

Therefore:

```
Invalid
```

17. Valid Parentheses Algorithm

```
def is_valid(s):
    stack = []

    pairs = {
        ')': '(',
        ']': '[',
        '}': '{'
    }

    for ch in s:
        if ch in "([{":
            stack.append(ch)

        elif ch in ")]}":
            if not stack:
                return False

            if stack.pop() != pairs[ch]:
                return False

    return len(stack) == 0
```

Complexity:

```
Time: 0(n)
Space: 0(n)
```

18. Monotonic Stack

A monotonic stack is one of the most important advanced stack patterns.

It maintains elements in either:

Increasing order

```
1
3
5
7
```

or:

Decreasing order

9
7
5
3

The stack is maintained according to a monotonic property.

19. Why Monotonic Stacks?

They are useful when a problem asks for:

- Next greater element
- Next smaller element
- Previous greater element
- Previous smaller element
- Daily temperatures
- Stock span
- Largest rectangle in histogram

Instead of repeatedly searching to the left/right, we can often solve these problems in:

$O(n)$

20. Next Greater Element

Given:

[2, 1, 5, 3, 4]

For each element, find the first greater element to its right.

Answer:

2 → 5
1 → 5
5 → -1
3 → 4
4 → -1

Result:

[5, 5, -1, 4, -1]

A monotonic stack can solve this in $O(n)$.

21. Queue

A Queue is a linear data structure where insertion happens at the rear and deletion happens from the front.

The fundamental principle is:

FIFO — First In, First Out

The element inserted first is removed first.

22. Real-World Queue

Think about people standing in a line.

```

Front                Rear
↓                    ↓
[Person A] [Person B] [Person C]
  
```

Person A arrived first.

Therefore A leaves first.

```
Dequeue()
```

removes:

```
Person A
```

23. Queue Operations

Operation	Meaning	Typical Time
enqueue(x)	Insert x at rear	O(1)
dequeue()	Remove from front	O(1)
front()	View front element	O(1)
rear()	View rear element	O(1)
isEmpty()	Check empty	O(1)
size()	Number of elements	O(1)

24. Queue Example

Initially:

```
EMPTY
```

Enqueue:

```
enqueue(10)
```

```

Front
↓
[10]
↑
Rear
  
```

Then:

```
enqueue(20)
enqueue(30)
```

```
Front      Rear
  ↓          ↓
[10] → [20] → [30]
```

Now:

```
dequeue()
```

removes 10.

```
Front      Rear
  ↓          ↓
[20] → [30]
```

Then:

```
dequeue()
```

removes 20.

25. Queue Overflow

For a fixed-size queue:

```
Capacity = 3
[10][20][30]
```

Trying:

```
enqueue(40)
```

causes:

```
QUEUE OVERFLOW
```

26. Queue Underflow

If:

```
Queue = EMPTY
```

and we call:

```
dequeue()
```

we get:

```
QUEUE UNDERFLOW
```

27. Queue Using Python

Python's `collections.deque` is ideal for queues.

```
from collections import deque

queue = deque()

queue.append(10)
queue.append(20)
queue.append(30)

print(queue.popleft()) # 10
print(queue.popleft()) # 20
```

Queue:

```
[10, 20, 30]
```

After two removals:

```
[30]
```

28. Why Not Use a Python List for Queue?

You could write:

```
queue = []

queue.append(10)
queue.append(20)
queue.append(30)

queue.pop(0)
```

But `pop(0)` is $O(n)$ because the remaining elements need to be shifted.

Example:

```
[10, 20, 30, 40, 50]
  ↑
remove
```

After removal:

```
[20, 30, 40, 50]
```

Elements have to move.

With:

```
deque.popleft()
```

the operation is designed for efficient removal from the left.

Typical complexity:

```
deque.append() →  $O(1)$ 
deque.popleft() →  $O(1)$ 
```

29. Circular Queue

A normal array queue can waste space.

Consider:

```
Capacity = 5
```

```

[10][20][30][40][50]
  ↑       ↑
front    rear

```

Remove two:

```

dequeue()
dequeue()

```

Conceptually:

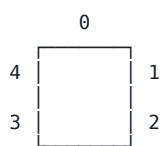
```

[ ][ ][30][40][50]
  ↑       ↑
front    rear

```

There is now free space at the beginning.

A circular queue allows the rear to wrap around and reuse this space.



After reaching the last index:

```

rear = (rear + 1) % capacity

```

wraps it back to index 0.

30. Circular Queue Formula

For a circular queue:

```

rear = (rear + 1) % capacity

```

and:

```

front = (front + 1) % capacity

```

This is an important interview concept.

31. Queue Using Linked List

A queue can be implemented using a linked list with:

```

front
  ↓
[10] → [20] → [30] → None
                        ↑
                      rear

```

Enqueue

Insert at rear:

```

[10] → [20] → [30] → [40]
                        ↑
                      rear

```

Dequeue

Remove from front:

```
[20] → [30] → [40]
  ↑
front
```

With both front and rear pointers:

```
enqueue → O(1)
dequeue → O(1)
```

32. Applications of Queue

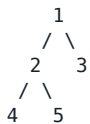
Queues are used in:

1. CPU scheduling
2. Printer scheduling
3. Network packet processing
4. Breadth-First Search
5. Level-order traversal
6. Message queues
7. Task scheduling
8. Operating systems
9. Producer-consumer systems
10. Request processing
11. Traffic simulation
12. Buffering

33. BFS and Queue

One of the most important DSA applications.

Suppose we have:



Breadth-first traversal:

```
1 → 2 → 3 → 4 → 5
```

We use a queue.

Initially:

```
Queue:
[1]
```

Remove 1:

Queue:
[2, 3]

Remove 2:

Queue:
[3, 4, 5]

Remove 3:

Queue:
[4, 5]

And so on.

This naturally follows:

FIFO

34. BFS Implementation

```
from collections import deque

def bfs(graph, start):
    queue = deque([start])
    visited = {start}

    while queue:
        node = queue.popleft()

        print(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Typical complexity:

Time: $O(V + E)$
Space: $O(V)$

where:

- V = number of vertices
- E = number of edges

35. Deque

Deque stands for:

Double-Ended Queue

It allows insertion and deletion from both ends.

```
Front                                Rear
↓                                  ↓
[10] [20] [30] [40]
```

↑
remove/add

↑
remove/add

Operations include:

```
append()
appendleft()
pop()
popleft()
```

Python:

```
from collections import deque

dq = deque()

dq.append(10)
dq.append(20)

dq.appendleft(5)

print(dq)
```

Result:

```
[5, 10, 20]
```

36. Stack vs Queue

This is one of the most important comparisons.

Feature	Stack	Queue
Principle	LIFO	FIFO
Insertion	Top	Rear
Removal	Top	Front
Main operation	Push	Enqueue
Removal operation	Pop	Dequeue
Used in DFS	Yes	No
Used in BFS	No	Yes
Recursion	Yes	No
Scheduling	Sometimes	Very common
Parentheses	Yes	No

37. Stack vs Queue Visualization

Stack

```
TOP
↓
[30] ← pop
[20]
[10]
```

Last inserted:

30

comes out first.

LIFO

Queue

FRONT REAR
 ↓ ↓
 [10] → [20] → [30]
 ↑
 dequeue

First inserted:

10

comes out first.

FIFO

38. Important DSA Pattern: Stack vs Queue

When you see:

"Most recently added"

Think:

Stack

"First added"

Think:

Queue

"Explore level by level"

Think:

Queue / BFS

"Explore as deep as possible"

Think:

Stack / DFS

"Matching brackets"

Think:

Stack

"Next greater/smaller"

Think:

Monotonic Stack**"Tasks processed in arrival order"**

Think:

Queue

39. Complexity Cheat Sheet

Stack

Operation	Complexity
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
Search	$O(n)$
Access arbitrary element	$O(n)$
Space	$O(n)$

Queue

Operation	Complexity
Enqueue	$O(1)$
Dequeue	$O(1)$
Front	$O(1)$
Rear	$O(1)$
Search	$O(n)$
Space	$O(n)$

40. Common Interview Problems — Stack

You should eventually solve these in roughly this order:

Beginner

1. Valid Parentheses
2. Implement Stack Using Array

3. Implement Stack Using Linked List
4. Min Stack
5. Remove All Adjacent Duplicates

Intermediate

1. Next Greater Element
2. Daily Temperatures
3. Evaluate Reverse Polish Notation
4. Decode String
5. Simplify Path
6. Asteroid Collision
7. Evaluate Expression

Advanced

1. Largest Rectangle in Histogram
2. Maximal Rectangle
3. Trapping Rain Water
4. Basic Calculator
5. Remove K Digits
6. Sum of Subarray Minimums

41. Common Interview Problems — Queue

Beginner

1. Implement Queue Using Array
2. Implement Queue Using Linked List
3. Implement Queue Using Two Stacks
4. Implement Stack Using Two Queues

Intermediate

1. Number of Recent Calls
2. Design Circular Queue
3. Moving Average from Data Stream
4. First Unique Character / Streaming variants
5. Binary Tree Level Order Traversal
6. Rotting Oranges

Advanced

1. Sliding Window Maximum
2. Shortest Path in Grid
3. 0-1 BFS
4. Task Scheduling
5. Design Hit Counter

42. Stack Using Two Queues

An interesting interview problem is:

Implement a stack using queues.

Normally:

Stack → LIFO
Queue → FIFO

We need to simulate LIFO behavior using FIFO structures.

One approach:

```
push(1)
push(2)
push(3)
```

Queue:

```
[1, 2, 3]
```

To pop 3, rearrange the queue:

```
[3, 1, 2]
```

Then remove 3.

This demonstrates an important interview principle:

A data structure can be implemented using another data structure by manipulating its operations.

43. Queue Using Two Stacks

Another famous interview problem:

Implement a queue using two stacks.

Use:

```
stack_in
stack_out
```

For:

```
enqueue(1)
```

```
enqueue(2)
enqueue(3)
```

we have:

```
stack_in:
3
2
1
```

To dequeue, transfer elements:

```
stack_out:
1
2
3
```

Now:

```
pop()
```

returns:

```
1
```

Therefore FIFO behavior is achieved using two LIFO structures.

44. Amortized $O(1)$

This is an important interview concept.

For a queue implemented using two stacks, an individual operation may sometimes require:

```
 $O(n)$ 
```

because elements are transferred.

But each element is transferred only a limited number of times.

Therefore the amortized complexity of enqueue/dequeue can be:

```
 $O(1)$ 
```

This concept becomes important in more advanced DSA interviews.

45. Monotonic Queue

Just as we have a monotonic stack, we can have a:

Monotonic Queue

It maintains elements in increasing or decreasing order.

A classic application is:

Sliding Window Maximum

Example:

```
nums = [1,3,-1,-3,5,3,6,7]
window = 3
```

Windows:

```
[1,3,-1] → 3
[3,-1,-3] → 3
[-1,-3,5] → 5
[-3,5,3] → 5
[5,3,6] → 6
[3,6,7] → 7
```

Answer:

```
[3,3,5,5,6,7]
```

A monotonic deque can solve this in:

$O(n)$

instead of:

$O(n \times k)$

with a naive approach.

46. Key Mental Models

Memorize these.

Stack

```
TOP
↓
[C]
[B]
[A]
```

```
push → TOP
pop ← TOP
```

Rule:

Last In → First Out

Queue

```
FRONT                                REAR
↓                                    ↓
[A] → [B] → [C]
dequeue ←                            enqueue →
```

Rule:

First In → First Out

47. Stack Problem Recognition

When reading a DSA problem, look for these clues:

"previous"
"next greater"
"next smaller"
"nearest greater"
"nearest smaller"
"matching"
"balanced"
"undo"
"reverse"
"nested"
"expression"
"histogram"
"monotonic"

These frequently indicate a stack.

48. Queue Problem Recognition

Look for:

"first come first served"
"level order"
"BFS"
"shortest path in unweighted graph"
"arrival order"
"process requests"
"sliding window"
"stream"
"tasks"
"scheduling"

These frequently indicate a queue/deque.

49. Most Important Things to Memorize

For interviews, make sure these are automatic:

Stack

LIFO
push
pop
peek
overflow
underflow
monotonic stack

Queue

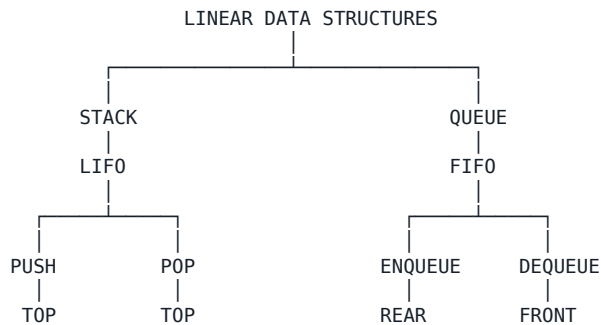
FIFO
enqueue
dequeue
front
rear
circular queue
deque

Algorithms

Stack → DFS
Queue → BFS

Stack → Parentheses
 Stack → Next Greater Element
 Stack → Histogram
 Deque → Sliding Window Maximum
 Queue → Level Order Traversal

50. Final Cheat Sheet



Stack

push(x) → O(1)
 pop() → O(1)
 peek() → O(1)

Main applications:

Recursion
 DFS
 Parentheses
 Undo/Redo
 Expression evaluation
 Backtracking
 Monotonic stack

Queue

enqueue(x) → O(1)
 dequeue() → O(1)
 front() → O(1)

Main applications:

BFS
 Level-order traversal
 Scheduling
 Buffering
 Networking
 Task processing
 Producer-consumer

Deque

append() → O(1)
 appendleft() → O(1)
 pop() → O(1)
 popleft() → O(1)

Main application:

Sliding Window Maximum

The one-line memory trick

Stack = Last person enters the elevator → first person gets out.

Queue = First person enters the line → first person gets served.

For your Google/Microsoft-style DSA preparation, the next important step after these fundamentals is to learn Stack/Queue problem-solving patterns, especially Valid Parentheses → Min Stack → Monotonic Stack → Next Greater Element → Daily Temperatures → Histogram → Queue using Two Stacks → BFS → Monotonic Deque/Sliding Window Maximum.

USER

Create a professional textbook-style PDF from the entire conversation above on DSA algos, preserving every single word exactly as given. Instructions:

1. Preserve Content Exactly
 - Include 100% of the conversation content.
 - Do not remove, summarize, paraphrase, rewrite, shorten, or modify any text.
 - Preserve all questions, answers, code snippets, examples, explanations, formulas, tables, and notes exactly as given.
 - Maintain the original sequence and context of the conversation.
 - Ensure not a single word is omitted.
2. Convert into Textbook Format
 - Create a professional title page.
 - Generate a complete table of contents.
 - Organize the content into logical chapters, sections, and subsections based on topics discussed.
 - Use chapter numbering and hierarchical headings.
 - Add page numbers, headers, and footers.
3. Academic Presentation
 - Format content like a university-level textbook.
 - Highlight important concepts, definitions, theorems, formulas, and takeaways using callout boxes.
 - Present code examples in properly formatted code blocks.
 - Format equations and mathematical expressions clearly.
 - Use professional typography, spacing, and layout standards.
4. Visual Enhancements
 - Create relevant diagrams, flowcharts, mind maps, concept maps, and illustrations where applicable.
 - Add topic-wise visual summaries.
 - Include chapter divider pages.
 - Use clean and readable educational design elements.
5. Learning Support
 - Add chapter summaries at the end of each chapter.
 - Generate key points and revision notes without altering the original conversation text.
 - Include glossary and index sections if applicable.
6. PDF Quality Requirements
 - Produce a print-ready, high-resolution PDF.
 - Ensure proper margins, alignment, and readability.
 - Keep formatting consistent throughout the document.
 - Ensure all images, charts, and diagrams are properly positioned.
7. Final Verification
 - Verify that every word from the original conversation has been included.
 - Verify that no content has been lost during restructuring.
 - Verify that all code blocks and examples remain unchanged.
 - Ensure the final PDF is complete, accurate, and professionally formatted.

Output:

Generate a polished textbook-style PDF titled:

"Complete Textbook Notes on DSA algos"

The final PDF must contain the entire conversation verbatim while presenting it in a structured, professional textbook format.

Part II — Supplementary Learning Support

This part is newly generated learning support. It is intentionally separate from the verbatim conversation so the original content remains unchanged.

CHAPTER ear

Foundations: Linear Data Structures

A linear data structure arranges elements sequentially. Stacks and queues restrict where insertion and deletion can occur, creating predictable processing behavior.

KEY TAKEAWAYS

Stack follows LIFO. • Queue follows FIFO. • The key distinction is the order in which elements leave the structure.

CHAPTER ns a

Stack: Core Operations and Implementations

A stack exposes its top as the active end. Push inserts there, pop removes there, and peek observes there. Array and linked-list implementations can provide constant-time basic operations.

KEY TAKEAWAYS

Know push, pop, peek, isEmpty, and size. • Fixed-size arrays require overflow handling. • Empty stacks require underflow handling.

CHAPTER s an

Stack Patterns and Applications

Stacks naturally model nested or reverse-order behavior. This makes them useful for call stacks, recursion, bracket matching, expression processing, DFS, backtracking, and monotonic-stack problems.

KEY TAKEAWAYS

Balanced parentheses is a canonical stack problem. • Monotonic stacks reduce repeated directional searches to linear-time patterns in many cases. • Recognize next/previous greater or smaller element language.

CHAPTER
ns

Queue: Core Operations and Implementations

A queue separates the insertion end from the deletion end. Enqueue operates at the rear, while dequeue operates at the front. Circular queues reuse array capacity efficiently.

KEY TAKEAWAYS

Know enqueue, dequeue, front, rear, isEmpty, and size. • Use a deque for efficient double-ended operations. • Circular queues use modular index movement.

CHAPTER
S, a

Queues, BFS, and Deques

Queues are fundamental to breadth-first exploration because nodes are processed in the same order in which they enter the frontier. A deque extends the queue abstraction to both ends.

KEY TAKEAWAYS

BFS typically uses a queue. • Level-order tree traversal is a direct queue application. • Monotonic deques are central to sliding-window maximum problems.

CHAPTER Patterns

Interview Patterns and Complexity

Interview problems often test recognition rather than memorization alone. The words in a problem statement can suggest the appropriate structure and algorithmic pattern.

KEY TAKEAWAYS

Most recently added → think stack. • First added or level-by-level → think queue. • Next/previous greater or smaller → investigate a monotonic stack. • Sliding-window maximum → investigate a monotonic deque.

Visual Summary — Stack and Queue

STACK — LIFO	QUEUE — FIFO
TOP ↓ [30] [20] [10] push → TOP pop ← TOP	FRONT REAR ↓ ↓ [10] → [20] → [30] ↑ dequeue ← enqueue →

Pattern Recognition Map

Problem clue	Likely tool / pattern
Most recently added	Stack
First added / arrival order	Queue
Level by level	Queue / BFS
Deep exploration	Stack / DFS
Matching brackets	Stack
Next/previous greater or smaller	Monotonic Stack
Sliding window maximum	Monotonic Deque

Rapid Revision Sheet

Concept	Revision note
Stack	LIFO; insertion and deletion at top; push/pop/peek are typically $O(1)$.
Queue	FIFO; insertion at rear and deletion at front; enqueue/dequeue are typically $O(1)$.
Deque	Double-ended queue; efficient insertion/deletion from both ends.
Overflow	Attempt to add beyond a fixed capacity.
Underflow	Attempt to remove from an empty structure.
Monotonic Stack	Maintains increasing or decreasing order to solve directional-neighbor problems efficiently.
Circular Queue	Reuses array space by wrapping indices with modulo arithmetic.
BFS	Breadth-first traversal; queue-based; typical graph complexity $O(V + E)$.

Recommended Practice Progression

Valid Parentheses → Min Stack → Monotonic Stack → Next Greater Element → Daily Temperatures → Largest Rectangle in Histogram → Queue using Two Stacks → BFS → Monotonic Deque / Sliding Window Maximum

Glossary

Term	Meaning
LIFO	Last In, First Out.
FIFO	First In, First Out.
Push	Insert an element onto the top of a stack.
Pop	Remove the top element from a stack.
Peek	Inspect the top element without removing it.
Enqueue	Insert an element at the rear of a queue.
Dequeue	Remove an element from the front of a queue.
Front	The deletion/access end of a queue.
Rear	The insertion end of a queue.
Deque	Double-ended queue supporting operations at both ends.
Monotonic Stack	A stack maintained in increasing or decreasing order.
Circular Queue	A queue whose array indices wrap around to reuse capacity.
BFS	Breadth-First Search, normally implemented with a queue.
DFS	Depth-First Search, commonly implemented with recursion or a stack.
Amortized Complexity	Average cost per operation over a sequence of operations.

Index

- Amortized $O(1)$
- Array Stack
- Balanced Parentheses
- BFS
- Circular Queue
- Deque
- DFS
- FIFO
- Function Calls
- LIFO
- Linked List Stack
- Monotonic Queue
- Monotonic Stack
- Next Greater Element
- Overflow
- Parentheses Matching
- Peek
- Pop
- Push
- Queue
- Queue Using Two Stacks
- Recursion
- Sliding Window Maximum
- Stack
- Stack Using Two Queues
- Underflow
- Valid Parentheses

Final Verification

The PDF was generated from the conversation text represented in this chat context. The original user requests and assistant response are included in Part I. The assistant's Markdown headings, code fences, tables, examples, formulas, and explanatory wording are retained in the verbatim source representation; the additional textbook material is clearly separated in Part II.

INTEGRITY NOTE

The original conversation is preserved as a dedicated verbatim section. Supplementary summaries and revision aids are additive and are not substituted for the original text.